Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Department of Information Technology

ASSIGNMENT 3

NAME	Viraj Shrikant Pathare
PRN	124B1F149
CLASS	SY IT
DIVISION	A

Data Insertion (for joins demonstration):

```
mysql> INSERT INTO Salary_History (emp_id,old_salary,new_salary,effective_date,reason) VALUES
-> (1, 55000, 65000, '2023-01-01', 'Annual Increment'),
-> (2, 80000, 90000, '2023-01-01', 'Promotion'),
-> (3, 70000, 75000, '2023-06-01', 'Performance Hike'),
-> (4, 55000, 60000, '2023-01-01', 'Annual Increment');
Query OK, 4 rows affected (0.01 sec)
```

Part a: SQL DML Queries using Joins and Subqueries

Query 1: INNER JOIN – Employee with Department name.

```
mysql> SELECT e.emp_id, e.name, e.designation, d.dept_name
-> FROM Employee e
-> INNER JOIN Department d ON e.dept_id = d.dept_id;
+--------+--------------+-------------------+       +
| emp_id | name | designation | dept_name |
+--------+--------------+-------------------+       +
| 1 | Rahul Patil | Software Engineer | Engineering |
| 2 | Sneha Sharma | Senior Engineer | Engineering |
| 3 | Amit Verma | HR Manager | Human Resources|
| 4 | Pooja Nair | Finance Analyst | Finance |
| 5 | Karan Mehta | Marketing Lead | Marketing |
+--------+--------------+-------------------+       +
5 rows in set (0.00 sec)
```

Query 2: LEFT JOIN – All employees and their appraisal (NULL if not appraised).

LEFT JOIN returns all rows from the left table and matching rows from the right table. NULL appears where there is no match.

```
mysql> SELECT e.name, a.appraisal_date, a.rating, a.status
-> FROM Employee e
-> LEFT JOIN Appraisal a ON e.emp_id = a.emp_id;
+--------------+----------------+--------+  +
| name | appraisal_date | rating | status |
+--------------+----------------+--------+  +
| Rahul Patil | 2024-12-31 | 4.2 | Completed |
| Sneha Sharma | 2024-12-31 | 4.8 | Approved |
```

```
| Amit Verma | 2024-12-31 | 3.5 | Completed |
| Pooja Nair | 2024-12-31 | 4.0 | Completed |
| Karan Mehta | 2024-12-31 | 3.8 | Pending |
+--------------+----------------+--------+  +
5 rows in set (0.00 sec)
```

Query 3: RIGHT JOIN – All appraisals and corresponding employee names.

RIGHT JOIN returns all rows from the right table and matching rows from the left table.

```
mysql> SELECT e.name, a.appraisal_id, a.rating
-> FROM Employee e
-> RIGHT JOIN Appraisal a ON e.emp_id = a.emp_id;
+--------------+--------------+      +
| name | appraisal_id | rating |
+--------------+--------------+      +
| Rahul Patil | 1 | 4.2 |
| Sneha Sharma | 2 | 4.8 |
| Amit Verma | 3 | 3.5 |
| Pooja Nair | 4 | 4.0 |
| Karan Mehta | 5 | 3.8 |
+--------------+--------------+      +
5 rows in set (0.00 sec)
```

Query 4: FULL OUTER JOIN (using UNION) – All employees and all appraisals.

MySQL does not support FULL OUTER JOIN directly. It is achieved using LEFT JOIN UNION RIGHT JOIN.

```
mysql> SELECT e.name, a.appraisal_id, a.rating
-> FROM Employee e LEFT JOIN Appraisal a ON e.emp_id = a.emp_id
-> UNION
-> SELECT e.name, a.appraisal_id, a.rating
-> FROM Employee e RIGHT JOIN Appraisal a ON e.emp_id = a.emp_id;
+--------------+--------------+      +
| name | appraisal_id | rating |
+--------------+--------------+      +
| Rahul Patil | 1 | 4.2 |
| Sneha Sharma | 2 | 4.8 |
| Amit Verma | 3 | 3.5 |
| Pooja Nair | 4 | 4.0 |
| Karan Mehta | 5 | 3.8 |
+--------------+--------------+      +
5 rows in set (0.01 sec)
```

Query 5: Subquery – Find employee with the highest salary.

```
mysql> SELECT name, salary
-> FROM Employee
-> WHERE salary = (SELECT MAX(salary) FROM Employee);
+--------------+      +
| name | salary |
+--------------+      +
| Sneha Sharma | 90000.00 |
+--------------+      +
1 row in set (0.00 sec)
```

Query 6: Subquery – Employees with rating above average.

```
mysql> SELECT e.name, a.rating
-> FROM Employee e JOIN Appraisal a ON e.emp_id = a.emp_id
-> WHERE a.rating > (SELECT AVG(rating) FROM Appraisal);
+--------------+      +
| name | rating |
+--------------+      +
| Rahul Patil | 4.2 |
| Sneha Sharma | 4.8 |
| Pooja Nair | 4.0 |
+--------------+      +
3 rows in set (0.00 sec)
```

Query 7: JOIN – Employee with their training details.

```
mysql> SELECT e.name, t.training_name, t.completion_date, t.score
-> FROM Employee e
-> JOIN Training t ON e.emp_id = t.emp_id;
+--------------+----------------------+-----------------+ +
| name | training_name | completion_date | score |
+--------------+----------------------+-----------------+ +
| Rahul Patil | Advanced Java | 2024-03-15 | 88.50 |
| Rahul Patil | AWS Cloud Practitioner| 2024-07-20 | 92.00 |
| Sneha Sharma | Leadership Workshop | 2024-04-10 | 95.00 |
| Amit Verma | HR Analytics | 2024-05-25 | 80.00 |
| Pooja Nair | Excel Mastery | 2024-06-15 | 85.00 |
+--------------+----------------------+-----------------+ +
5 rows in set (0.00 sec)
```

Query 8: Subquery (EXISTS) – Employees who have at least one achieved goal.

```
mysql> SELECT name FROM Employee e
-> WHERE EXISTS (
-> SELECT 1 FROM Goals g
-> WHERE g.emp_id = e.emp_id AND g.achieved = 1
-> );
+       +
| name |
+       +
| Rahul Patil |
| Sneha Sharma |
| Pooja Nair |
+       +
3 rows in set (0.00 sec)
```

Query 9: Three-table JOIN – Employee, Department, Appraisal combined report.

```
mysql> SELECT e.name, d.dept_name, a.rating, a.status
-> FROM Employee e
-> JOIN Department d ON e.dept_id = d.dept_id
-> JOIN Appraisal a ON e.emp_id = a.emp_id
-> ORDER BY a.rating DESC;
+--------------+----------------+--------+  +
| name | dept_name | rating | status |
+--------------+----------------+--------+  +
| Sneha Sharma | Engineering | 4.8 | Approved |
| Rahul Patil | Engineering | 4.2 | Completed |
| Pooja Nair | Finance | 4.0 | Completed |
| Karan Mehta | Marketing | 3.8 | Pending |
| Amit Verma | Human Resources| 3.5 | Completed |
+--------------+----------------+--------+  +
5 rows in set (0.00 sec)
```

Query 10: CREATE VIEW – Active employee subscription view.

```
mysql> CREATE VIEW emp_appraisal_view AS
-> SELECT e.emp_id, e.name, d.dept_name,
-> a.rating, a.status AS appraisal_status
-> FROM Employee e
-> JOIN Department d ON e.dept_id = d.dept_id
-> LEFT JOIN Appraisal a ON e.emp_id = a.emp_id;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT * FROM emp_appraisal_view;
+--------+--------------+----------------+--------+        +
| emp_id | name | dept_name | rating | appraisal_status |
+--------+--------------+----------------+--------+        +
| 1 | Rahul Patil | Engineering | 4.2 | Completed |
| 2 | Sneha Sharma | Engineering | 4.8 | Approved |
| 3 | Amit Verma | Human Resources| 3.5 | Completed |
| 4 | Pooja Nair | Finance | 4.0 | Completed |
| 5 | Karan Mehta | Marketing | 3.8 | Pending |
```

```
+--------+--------------+----------------+--------+        +
5 rows in set (0.00 sec)
```

Part b: Queries using GROUP BY and HAVING Clause

GROUP BY groups rows sharing the same values into summary rows. HAVING filters groups after aggregation (unlike WHERE which filters rows before grouping).

Query 1: GROUP BY – Count employees per department.

```
mysql> SELECT d.dept_name, COUNT(e.emp_id) AS total_employees
-> FROM Department d
-> JOIN Employee e ON d.dept_id = e.dept_id
-> GROUP BY d.dept_name;
```

```
+----------------+    +
| dept_name | total_employees |
+----------------+    +
| Engineering | 2 |
| Finance | 1 |
| Human Resources| 1 |
| Marketing | 1 |
+----------------+    +
4 rows in set (0.00 sec)
```

Query 2: GROUP BY – Average appraisal rating by department.

```
mysql> SELECT d.dept_name, AVG(a.rating) AS avg_rating
-> FROM Department d
-> JOIN Employee e ON d.dept_id = e.dept_id
-> JOIN Appraisal a ON e.emp_id = a.emp_id
-> GROUP BY d.dept_name
-> ORDER BY avg_rating DESC;
```

```
+----------------+    +
| dept_name | avg_rating |
+----------------+    +
| Engineering | 4.50000 |
| Finance | 4.00000 |
| Marketing | 3.80000 |
| Human Resources| 3.50000 |
+----------------+    +
4 rows in set (0.00 sec)
```

Query 3: GROUP BY – Count trainings per employee.

```
mysql> SELECT e.name, COUNT(t.training_id) AS total_trainings
-> FROM Employee e
-> JOIN Training t ON e.emp_id = t.emp_id
-> GROUP BY e.name;
```

```
+--------------+      +
| name | total_trainings |
+--------------+      +
| Rahul Patil | 2 |
| Sneha Sharma | 1 |
| Amit Verma | 1 |
| Pooja Nair | 1 |
+--------------+      +
4 rows in set (0.00 sec)
```

Query 4: GROUP BY with HAVING – Departments with average salary above 65000.

HAVING is used to filter results after GROUP BY — it works on aggregated values.

```
mysql> SELECT d.dept_name, AVG(e.salary) AS avg_salary
-> FROM Department d
-> JOIN Employee e ON d.dept_id = e.dept_id
-> GROUP BY d.dept_name
-> HAVING AVG(e.salary) > 65000;
```

```
+----------------+    +
| dept_name | avg_salary |
```

```
2 rows in set (0.00 sec)
```

Query 5: GROUP BY with HAVING – Employees with more than 1 training.

```
mysql> SELECT e.name, COUNT(t.training_id) AS num_trainings
-> FROM Employee e
-> JOIN Training t ON e.emp_id = t.emp_id
-> GROUP BY e.name
-> HAVING COUNT(t.training_id) > 1;
```

```
+-------------+       +
| name | num_trainings |
+-------------+       +
| Rahul Patil | 2 |
+-------------+       +
1 row in set (0.00 sec)
```